

라즈베리파이 기반 다중 장치 제어 시스템

1. 프로젝트 개요

1.1 프로젝트 소개

본 프로젝트는 라즈베리파이를 기반으로 LED, 부저(Buzzer), 7세그먼트 디스플레이, CDS 조도 센서를 통합하여 제어하는 다중 장치 제어 시스템이다. HTTP 웹 서버를 구축하고 멀티스레드를 활용하여 다수의 클라이언트 요청을 동시에 처리할 수 있도록 설계하였다.

사용자는 웹 브라우저(Web UI) 또는 CLI 클라이언트 프로그램을 통해 라즈베리파이에 연결된 하드웨어 장치를 원격으로 제어할 수 있다.

1.2 사용 기술 스택

구분	기술	설명
서버 / CLI	C	HTTP 웹서버, 멀티스레드, 소켓 통신
하드웨어 제어	C + WiringPi	GPIO 제어, PWM, SoftTone
동적 라이브러리	C Shared Library (.so)	LED, CDS, Buzzer, Segment 기능 분리
Web UI	HTML / CSS / JavaScript	반응형 대시보드, 실시간 CDS 및 타이머 표시
버전 관리	Git	소스코드 버전 관리
UI 디자인	Figma	웹 UI 프로토타입 설계

1.3 시스템 구성

- 웹 서버 (webserver.c): HTTP GET 요청 파싱, 장치 제어 스레드 관리, 정적 파일 서빙
- 동적 라이브러리 (librasp.so): led.c / cds.c / buzzer.c / segment.c 개별 빌드 후 통합
- CLI 클라이언트 (client.c): 메뉴 기반 제어 인터페이스, CDS 로그 스레드 백그라운드 실행
- Web UI (index.html / style.css): 브라우저 기반 제어 패널, 1초 주기 CDS 실시간 갱신

2. 세부 구현 내용

2.1 웹 서버 (webserver.c)

2.1.1 데몬 프로세스

서버는 시작 시 `daemonize()` 함수를 통해 백그라운드 데몬 프로세스로 전환된다. `fork()` 후 부모 프로세스를 종료하고 `setsid()`로 세션 리더가 되며, 표준 입출력을 `/dev/null`로 리다이렉트한다. 워킹 디렉터리는 지정된 홈 경로로 설정된다.

2.1.2 멀티스레드 클라이언트 처리

`accept()` 루프에서 클라이언트 연결마다 `malloc`으로 소켓 `fd`를 힙에 할당하고 `pthread_create()`로 `clnt_connection` 스레드를 생성한다. `pthread_detach()`를 적용하여 스레드 종료 시 자동으로 리소스가 해제되도록 하였다.

2.1.3 HTTP 요청 파싱 및 라우팅

`strtok()`으로 요청 라인을 파싱하여 메소드와 경로를 분리한다. GET 메소드의 경로에 따라 각 장치 제어 함수를 호출하거나 파일(HTML/CSS)을 응답으로 전송한다.

LED와 CDS 는 동일한 하드웨어(PWM 핀)를 사용하므로 LED 제어 요청시 진행중인 CDS 스레드를 종료한 후 LED 스레드를 동작한다.

부저와 7세그먼트 또한 동일한 하드웨어(PWM 핀)를 사용하므로 각 요청 시 기존 실행중인 부저 혹은 7세그먼트 스레드를 먼저 종료한 후 새 스레드를 작동한다.

3.2 동적 라이브러리 (librasp.so)

LED, CDS, Buzzer, Segment 기능을 각각 별도의 .c 파일로 분리하여 개발하고, `aarch64-linux-gnu-gcc`로 크로스 컴파일하여 공유 라이브러리로 빌드하였다. 서버는 `dlopen()` / `dlsym()`으로 런타임에 함수를 로드하여 호출한다.

모듈	파일	기능
LED 제어	led.c	softPwmWrite로 밝기 단계 제어 (OFF/LOW/MED/HIGH)
CDS 센서	cds.c	조도 센서 읽기, 임계값 비교, 로그 파일 기록
부저	buzzer.c	softToneWrite로 부저 ON/OFF 제어
7세그먼트	segment.c	카운트다운 표시, 완료 시 부저 알림

3.3 CLI 클라이언트 (client.c)

메뉴 기반 인터페이스로 사용자 입력을 받아 HTTP GET 요청을 서버로 전송한다. 백그라운드 스레드가 1초 주기로 서버에서 CDS 데이터를 조회하여 로그 파일에 타임스탬프와 함께 기록한다.

3.4 Web UI

반응형 대시보드로 LED, Buzzer, CDS 센서, 7세그먼트 타이머를 카드 형태로 구성하였다. CDS 카드는 1초마다 CDS 값을 폴링하여 현재 조도값과 임계값, 활성화 상태를 실시간으로 갱신한다.

3. 개발 일정

일차	일정	주요 작업
1일차	주요 기능 개발	HTTP 웹서버 구현 (소켓, 스레드, HTTP 파싱) CLI 클라이언트 구현 (메뉴, 소켓 통신) 회로 설계 및 하드웨어

		연결 (LED, Buzzer, 7-Segment) WiringPi 기반 장치 제어 함수 개발 동적 라이브러리(librasp.so) 빌드 구성
2일차	Web UI 개발	Figma 기반 UI 디자인 설계 HTML/CSS 반응형 대시보드 구현 JavaScript fetch API 기반 장치 제어 연동 CDS 실시간 데이터 폴링 구현
3일차	코드 정리 및 리팩토링	코드 주석(Doxygen 스타일) 작성 LED 핸들러 중복 코드 통합 리팩토링 및 소켓 관리 개선 문제점 식별 및 보완

4. 문제점 및 보완사항

4.1 소켓 파일 디스크립터 고갈

문제

CLI 클라이언트의 백그라운드 로그 스레드가 1초마다 소켓을 새로 생성하는 구조에서, 서버 응답 지연 시 소켓이 닫히지 않고 누적되어 'Too many open files' 오류가 발생하였다.

보완

서버의 pthread_join을 pthread_detach로 교체하여 요청 처리 스레드가 블로킹 없이 독립 실행되도록 개선

클라이언트 send_request의 recv while 루프를 단일 recv 호출로 변경하여 응답 대기 시간 단축
소켓 생성 실패 시 exit 대신 sleep(1) 후 재시도로 변경하여 일시적 fd 고갈 상황에서 복구 가능하도록 처리

4.2 장치 동시 제어 시 충돌

문제

여러 장치를 동시에 제어할 경우 WiringPi GPIO 접근이 충돌하여 동작 중이던 장치가 비정상 종료되는 현상이 발생하였다. WiringPi는 내부적으로 thread-safe를 보장하지 않아 동시 접근 시 상태가 꼬이는 것이 원인이다.

보완

현재는 동시 제어를 피하는 방식으로 운용하며, mutex 락으로 해결이 가능할 가능성이 있다.

4.3 CDS 상태 UI 미동기화

문제

LED 제어 시 서버에서 CDS 스레드를 종료하고 로그 파일을 삭제하지만, 웹 UI는 이를 인지하지 못하고 이전 상태를 유지하는 현상이 발생하였다.

보완

CDS 값의 폴링 응답이 0,0일 때 UI를 Inactive 상태로 자동 갱신하도록 JavaScript 수정
서버의 LED 핸들러에서 CDS 기능 종료 시 로그 파일 즉시 삭제 처리 추가